

ENSF 380

TOPIC 22: I/O STREAMS

Terms

Stream

I/O Stream

Buffer

Flush

Java I/O Streams

Byte-based streams

- Read/Write data as 8-bit bytes
- Based on InputStream/OutputStream

Character-based streams

- Read/Write data as a sequence of characters
- Based on Reader/Writer

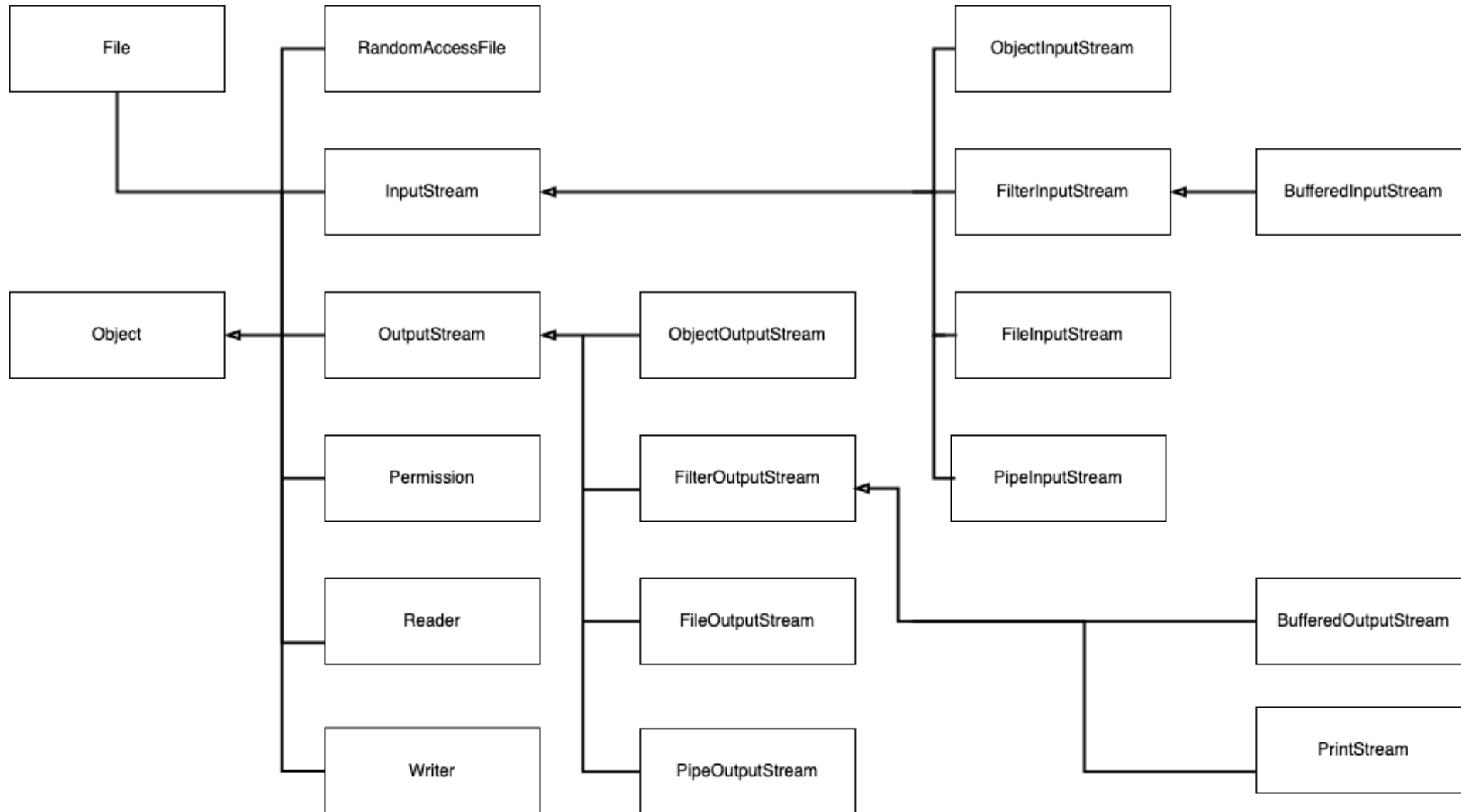
Random access

- Useful for direct access applications

ClassFile

- Used for retrieving information about a file or directory

Java I/O Classes



Byte-Based I/O Streams

Derived from InputStream/OutputStream

- FileInputStream – Read files
- FileOutputStream – Write files
- PipedInputStream and PipedOutputStream - Establish a data channel between threads
- FilterInputStream – Adds buffering
- FilterOutputStream – Adds buffering
 - PrintStream – Text output to a specific stream, provides `println()`

FileInputStream and FileOutputStream

01_ByteStream

```
import java.io.*;

...

FileInputStream in = null;
FileOutputStream out = null;

...

try {
    in = new FileInputStream(inName);
    out = new FileOutputStream(outName);
    int c;

    while ((c = in.read()) != -1) {
        out.write(c);
        System.out.print(c);
    }
}
```

FileInputStream and FileOutputStream

01_ByteStream

```
import java.io.*;

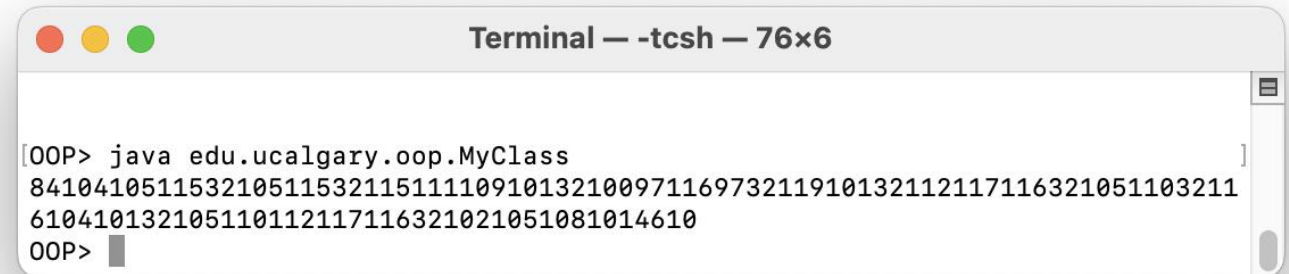
...

FileInputStream in = null;
FileOutputStream out = null;

...

try {
    in = new FileInputStream(inName);
    out = new FileOutputStream(outName);
    int c;

    while ((c = in.read()) != -1) {
        out.write(c);
        System.out.print(c);
    }
}
```



A terminal window titled "Terminal — -tcsh — 76x6" showing the execution of a Java program. The prompt is "[OOP> java edu.ucalgary.oop.MyClass" followed by the output of the program: "8410410511532105115321151111091013210097116973211910132112117116321051103211610410132105110112117116321021051081014610". The prompt is now "[OOP>]".

```
Terminal — -tcsh — 76x6

[OOP> java edu.ucalgary.oop.MyClass
8410410511532105115321151111091013210097116973211910132112117116321051103211
610410132105110112117116321021051081014610
[OOP> ]
```

FileInputStream and FileOutputStream

01_ByteStream

```
catch (FileNotFoundException e) {  
    System.out.println("Unable to find file " + inName);  
    e.printStackTrace();  
}  
  
catch (IOException e) {  
    System.out.println("I/O exception when trying to read " + inName  
        + "/write " + outName);  
    e.printStackTrace();  
}
```

FileInputStream and FileOutputStream

01_ByteStream

```
...
finally {
    if (out != null) {
        try {
            out.close();
        }
        catch (IOException e) {
            System.out.println("Couldn't close file " + outName);
            e.printStackTrace();
        }
    }
}
```

Exercise 22.1

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Buffered I/O Streams

BufferedInputStream

BufferedOutputStream

BufferedReader

BufferedWriter

```
inputStream = new BufferedReader(new FileReader("input.txt"));  
outputStream = new BufferedWriter(new FileWriter("output.txt"));
```

Buffered I/O Streams

03_Buffer

```
import java.io.*;

public class MyClass {
    public static void main(String[] args) throws Exception {
        var out = new PrintWriter(System.out);
        out.write("Manual flush.");
        Thread.sleep(2000);
        out.flush();
    }
}
```

Buffered I/O Streams

03_Buffer

```
public static void main(String[] args) throws Exception {  
    out.println();  
    out.printf("My cat is %d years old.", 11);  
    out.println();  
    Thread.sleep(2000);  
    out.close();  
}
```

Part of java.util

Used to easily write formatted data to a file

- Pass name of file to constructor
- If file does not exist, it is created
- If file exists, contents are discarded
- Method `format()` writes formatted text to file
- Method `close()` closes the file

Can throw exceptions

- `SecurityException` (no permission to write file)
- `FileNotFoundException` (file cannot be found; new file cannot be created)
- `FormatterClosedException` (attempt to write a closed object)

Part of java.util

Used to easily read data from a file

- Read data sequentially from a text file in the form of tokens
 - hasNext()
 - nextLong()
 - nextInt()
 - ...

Can throw exceptions

- FileNotFoundException
- IOException
- IllegalArgumentException
- IllegalStateException
- ...

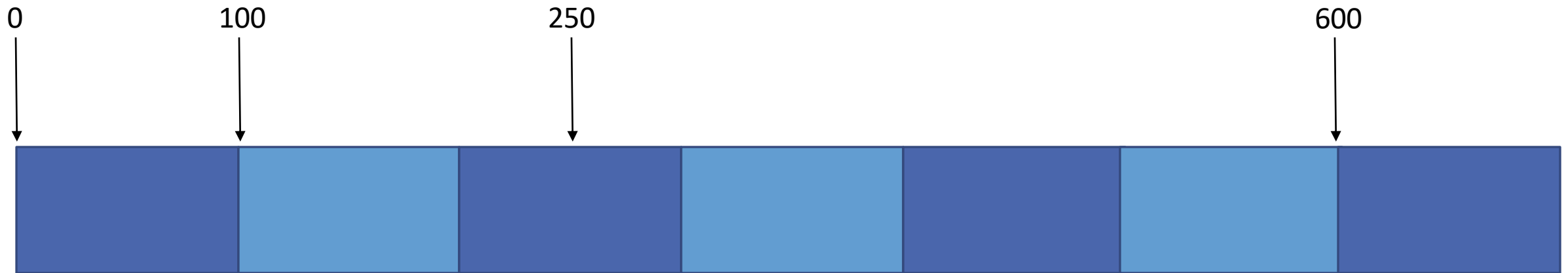
Random Access

Sequential files

- Can often not be modified without risk; they are overwritten
- Are inappropriate for instant-access applications

Random access

- Also called direct access
- Enables access at a specific location in a file



Random Access

```
public void start(char r) {
    try {
        file = new RandomAccessFile(fn, "r");
    }
    catch (IOException e) {
        System.err.println("Could not open file: " + fn);
        System.exit(1);
    }
}

public void start() {
    try {
        file = new RandomAccessFile(fn, "rw");
    }
    catch (IOException e) {
        System.err.println("Could not open file: " + fn);
        System.exit(1);
    }
}
```

Random Access

06_RandomAccess

```
public void writeMe() {  
    var str = new String("hi");  
  
    try {  
        file.writeInt(6);  
        file.writeDouble(4.01);  
        file.writeChars(str);  
    }  
  
    catch (IOException e) {  
        System.err.println("Could not write to file: " + fn);  
        this.end();  
        System.exit(1);  
    }  
}
```

Random Access

```
public int readMe() {
    int number = 0;
    double decimalNum = 0;
    int pointer = 0;

    try {
        number = file.readInt();
        decimalNum = file.readDouble();
        pointer = (int) file.getFilePointer();
    }

    System.out.printf("Int: %02d, Double: %.3f%n", number, decimalNum);
    return pointer;
}
```

Random Access

06_RandomAccess

```
public void seekMe(int offset) {
    char letters[] = new char[2];
    char temp;

    try {
        file.seek(offset);
        for (int count=0; count < letters.length; count++) {
            temp = file.readChar();
            letters[count] = temp;
        }
    }

    String str = String.valueOf(letters);
    System.out.printf("String: %s\n", str);
    return;
}
```

File

07_File

Useful for retrieving information about files and directories

Does not open files or provide file-processing capabilities

Has four constructors:

- `new File(pathAndName)`
- `new File(path, name)`
- `new File(fileObjWithPath, name)`
- `new File(uriObj)`

Supplies a number of methods:

- `boolean canRead()`
- `boolean canWrite()`
- `boolean exists()`
- `boolean getAbsolutePath()`
- ...

**Tip: Read the
documentation
on File**

File

07_File

Don't do this:

```
fullPath = "public\Lesson" + "\" + fileName);
```

```
fullPath = new File("public/Lesson", fileName);
```

Do this instead:

```
partialPath = new File("public", "Lesson");  
fullPath = new File(partialPath, fileName);
```

Important: You should never create file paths using String concatenation. Use File instead.

"Serialization"

```
public static ArrayList<String> readFile() throws IOException {  
    while (scan.hasNext()) {  
        tmp = scan.next();  
        // Token found, end of this section of content  
        if (tmp.equals("#")) {  
            allText.add(line);        // store line in arraylist  
            line = "";                // Reset line  
        }  
        // Divider not found, add token to line  
        } else {  
            line = line + " " + tmp;  
            tmp = "";  
        }  
    }  
}
```

"Serialization"

```
public static void writeFile(ArrayList<String> data) throws IOException {  
  
    String line = iter.next();  
    outFile.write(line);  
  
    while(iter.hasNext()) {  
        outFile.write(" # ");  
        line = iter.next();  
        outFile.write(line);  
    }  
  
}
```

"Serialization"

No provision for literal inclusion of the dividing token

- ...We could make it more robust by allowing \# to be a literal #
-Then we'd need to ensure \\ allowed a literal \

Requires knowledge of structure to use

Limited usage

Object Serialization

08_ObjSerialization

Object serialization

- Mechanism to read or write an entire object from a file
- `ObjectOutputStream.writeObject()`

Serialized object

- The object represented as a sequence of bytes, including data and type information

Deserialization

- Recreate an object in memory from data in a file
- `ObjectInputStream.readObject()`
- Can throw errors:
 - `EOFException` (attempt to read past end of file)
 - `ClassNotFoundException` (class for the object cannot be located)

Important: `java.io.Serial` was added in JDK 14 and adds annotations to serializing. Serializing is done with `java.io.Serializable`. Be careful that your IDE doesn't autocomplete import statements of `Serialize` to `Serial`.

Object Serialization

08_ObjSerialization

Serialization runtime associates a version number (serialVersionUID) with each serializable class

- Verify sender and receiver have loaded compatible classes for the object
- If the receiver loads object with different serialVersionUID, it will cause InvalidClassException

Update when object compatibility will be broken

```
class BasicClass implements Serializable {  
    static final long serialVersionUID = 3L;  
}
```

Exercise 22.2

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

```
public class Echo {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        System.out.println("Type something. Type 'quit' to exit:");  
  
        while (true) {  
            String input = scanner.nextLine();  
            if (input.equalsIgnoreCase("quit")) {  
                System.out.println("Goodbye!");  
                break;  
            }  
            System.out.println(input.toUpperCase());  
        }  
        scanner.close();  
    }  
}
```

Tip: Use the Scanner class to interact with the user when writing your project.

Exercise 22.3

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Important File I/O Reminders

Always use `try{} catch{}` to manage exceptions

- Opening, reading, writing, and closing all have the potential to throw exceptions
- Common exceptions are `FileNotFoundException` and `IOException`
- Exceptions can occur for many reasons: insufficient permissions, file does not exist or invalid path, file is locked, hardware or system issues, mismatch between file type and selected library

Ensure files are closed after access

- A `finally{}` block can be used to ensure closure
- Also check for exceptions when closing the file

In the absence of an absolute path, file locations are relative to the execution directory

File paths should be constructed using the `File` class to ensure cross-platform functioning

Summary: Knowledge

Different types of Java I/O Streams

Streams to files, screen

Differences between byte-based and char-based I/O Streams

Meaning of buffer, flush, and autoflush

Some of the challenges of internationalization

Summary: Skills

Use of the following classes:

- FileInputStream
- FileOutputStream
- FileReader
- FileWriter
- BufferedInputStream
- BufferedOutputStream
- BufferedReader
- BufferedWriter
- PrintWriter
- Formatter
- Scanner
- RandomAccessFile
- File

Use Thread.sleep()

How to (de)serialize objects

Additional Information

Related reading

- Volume 1, Chapter 2: Input and Output